

广域组网通信（二）

陈巍

2025年秋

本讲内容摘要

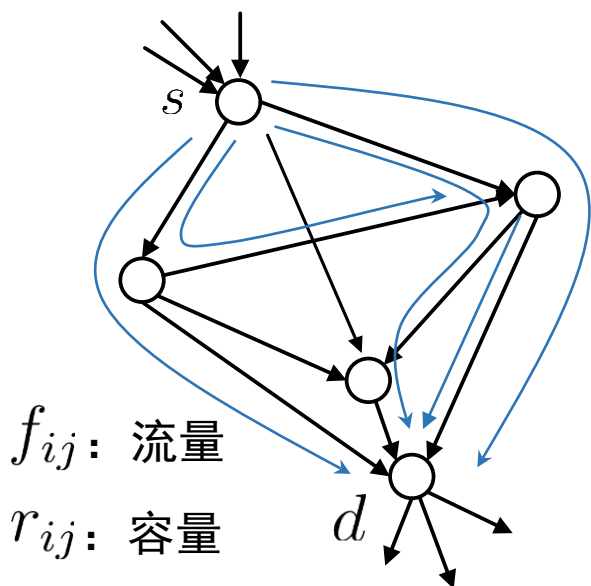
- ① 网络容量与准入控制；
- ② 弹性流与（分布式）流量控制；
- ③ 网络延时与负载均衡。

要求重点掌握：

- ① 最大流-最小割定理的应用；
- ② 端到端流量控制的机理与优化目标；
- ③ 排队延时的概率分析与最小延时路由。

网络容量与准入控制（一）

网络中任一对“源-宿”节点间的容量是有限的，（如常见的“电话占线”）。对于恒速业务流，如电话、视频，若不能满足其速率要求，则接纳其只会浪费带宽，故需要准入控制（Call Admission Control, CAC）。



有向有权图 $G = (V, E)$

记电话端局 s 处每路电话（或视频）的速率要求为1个单位。

问：该网络 G 最多能同时容纳多少路从源 s 到宿 d 的电话/视频？

注意：此时，不能采用最短路径（最小代价）路由，（因路径占用会改变其代价），而是采用多路径路由。

网络容量与准入控制（二）

定义：①记 S 为 G 中含源节点 s 但不含宿节点 d 的节点集，包含 d

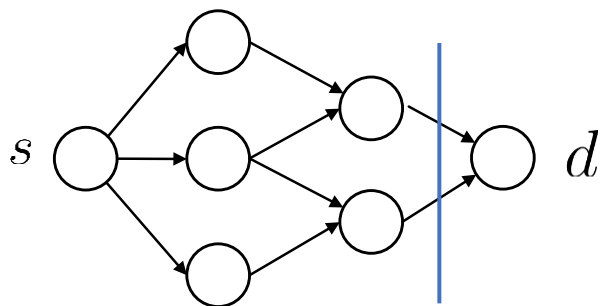
在内的 G 中其它节点集合记为 $\bar{S}$ ， $S \cup \bar{S} = V$

② 记 $E(S \rightarrow \bar{S})$ 为从 S 中节点到 $\bar{S}$ 中节点的边集合（不含 $\bar{S}$ 到 S 的边），
称一个割集。

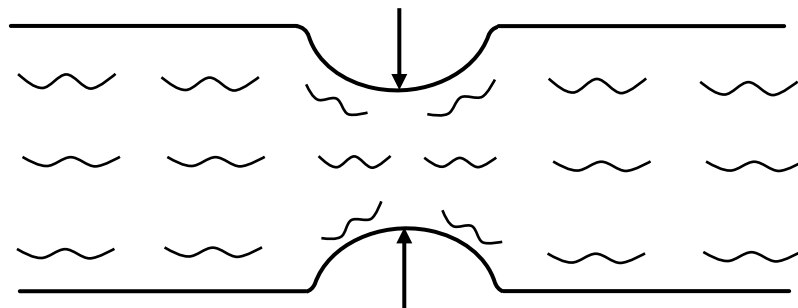
③ $E(S \rightarrow \bar{S})$ 中各边的容量总和称为割集容量，记 $r(S \rightarrow \bar{S})$

定理： G 中从 s 到 d 的最大流量等于从 s 到 d 的最小割集容量。

物理解释：水管的流量决定于最窄处的横截面积！



最小割为2



网络容量与准入控制（三）

Max-Flow-Min-Cut 定理并没有给出最大流的具体计算方法。这里我们给出一种贪婪算法——Ford-Fulkerson算法计算最大流。

定义：第 k 次迭代后的残存网络为 $G^k(V, E^k)$ ，其中， $r_{ij}^k = r_{ij} - f_{ij}$ （即链路 (i, j) 上使用流量 f_{ij} 后残存的容量（迭代过程中可以为负））

初始化： $f_{ij} \leftarrow 0, \quad \forall(i, j)$

当 （ G^k 中存在一条从 s 到 d 的路径 $\mathbf{P}$ ，且满足 $\forall(i, j) \in \mathbf{P}, r_{i,j}^k > 0$ ）

{

① 找出 $(i, j) \in \mathbf{P} \quad r^k(\mathbf{P}) = \min_{(i,j) \in \mathbf{P}} r_{ij}^k$

② for $f_{ij} \leftarrow f_{ij} + r^k(\mathbf{P});$

$f_{ji} \leftarrow f_{ji} - r^k(\mathbf{P});$

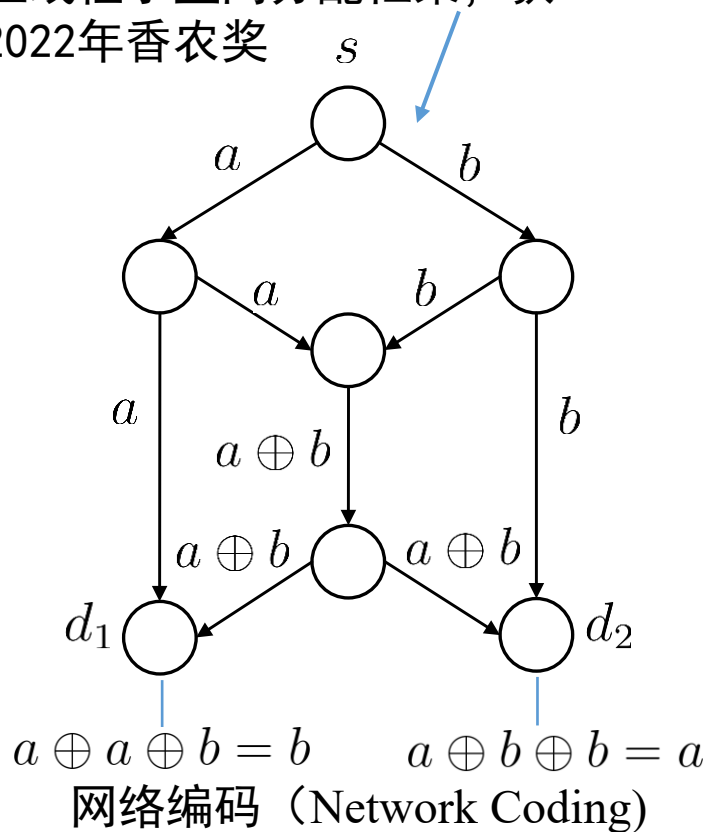
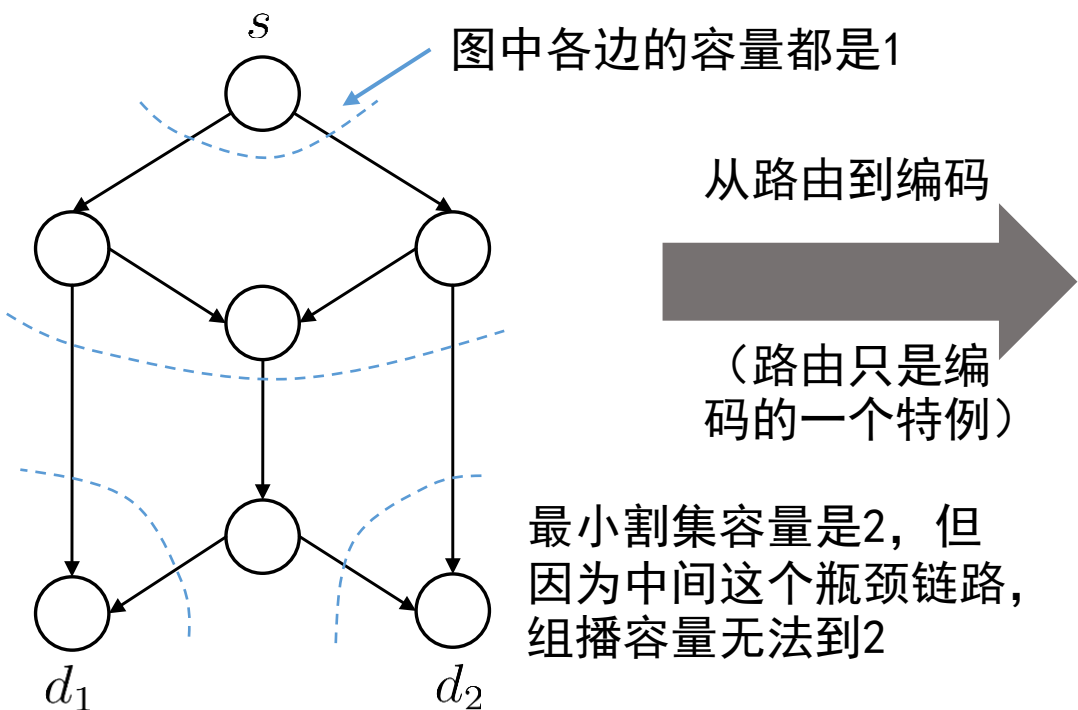
$k \leftarrow k + 1.$

} ←——当 G^k 中找不到从 s 到 d 的非零容量路径时停止

网络容量与准入控制（四）

视频会议网络中存在一种组播最大流问题，如下左图所示， s 向 d_1, d_2 组播相同的信息，此时最大流是多少？

杨伟豪因提出网络编码及其图上线性子空间分配框架，获2022年香农奖

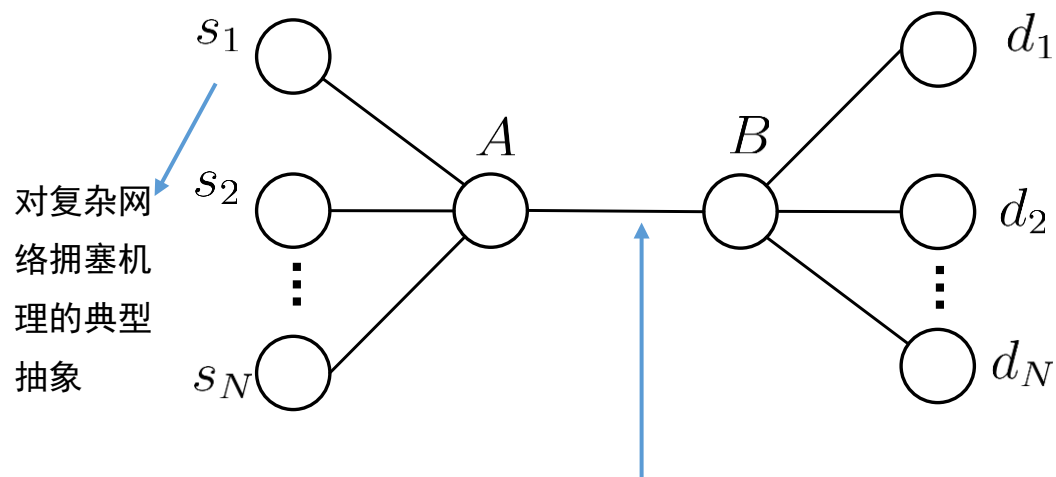


思考：组播的最小代价路由如何设计？

（最小生成树——树上总代价最小，而非根到每个节点各自代价最小）

面向弹性流的流量控制（一）

与语音、视频等业务“刚性”的速率要求不同，FTP下载等业务的速率要求是“弹性”的，“快更好，慢也可以等，总比没有接入强”。而且，像电话占线一样拒绝用户接入请求，让过一会再试，也不现实。故，我们研究用户以弹性方式共享广域网络资源，传输用户多，网络拥塞，带宽紧张时，每个人就都少传一点，典型拓扑如下“哑铃状”网络拓扑



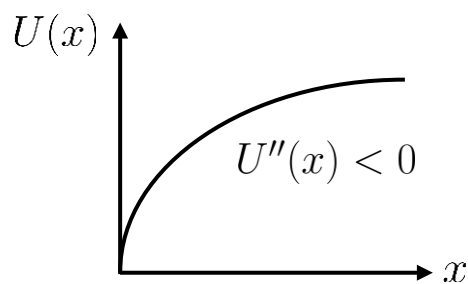
若所有链路容量接近，则中间为瓶颈链路

要求

- ①公平：用户对（瓶颈）带宽有差不多的优先权，分配的速率不应过于悬殊；
- ②要主动控制 $s_i \rightarrow A$ 的速率，否则A处因拥塞而大量丢包重传，浪费资源，延时激增。

面向弹性流的流量控制（二）

- 实际发展历史中，人们是先根据网络的各类工程约束和现象“拍脑袋”再反复尝试迭代得到的流量控制方法。20世纪末，F. Kelly等“逆向工程”给出了其一种理论解释，说明为什么有效和鲁棒。本课程以最简单的哑铃拓扑为例，介绍这种网络效用最大化（Network Utility Maximization, NUM）理论框架



$$\begin{aligned} \max \quad & \sum_i U(x_i) \\ \text{s.t.} \quad & \sum_i x_i \leq r \\ & x_i \geq 0, \forall i \end{aligned}$$

用户*i*的速率
速率非负

显然最优解为 $x^* = \frac{r}{N}$

问题：如何分布式求解且能动态适应s-d业务变化

瓶颈链路容量约束

复杂拓扑下会有多个瓶颈链路，则这一个线性约束拓展成一组即可，可行域为一个单纯形（多面体）

效用函数一般取二阶导小于0的，边际效用递减，以促进公平性，常取 $U(x) = \begin{cases} \ln x, & \alpha = 1 \\ (1 - \alpha)^{-1} x^{1-\alpha}, & \text{其他} \end{cases}$

面向弹性流的流量控制（三）

- 如下微分方程（动力系统）可以较好的逼近效用最大化问题的最优解

$$\frac{d}{dt}x_i(t) = p - x_i(t)q\mathbb{1}\left\{\sum_i x_i \geq r\right\}$$

瓶颈链路带宽价格（与供需关系有关）
也可从拉格朗日乘子法对偶松弛条件解释

- 物理解释：① $\sum_i x_i < r$ 时， $\frac{d}{dt}x_i(t) = p$

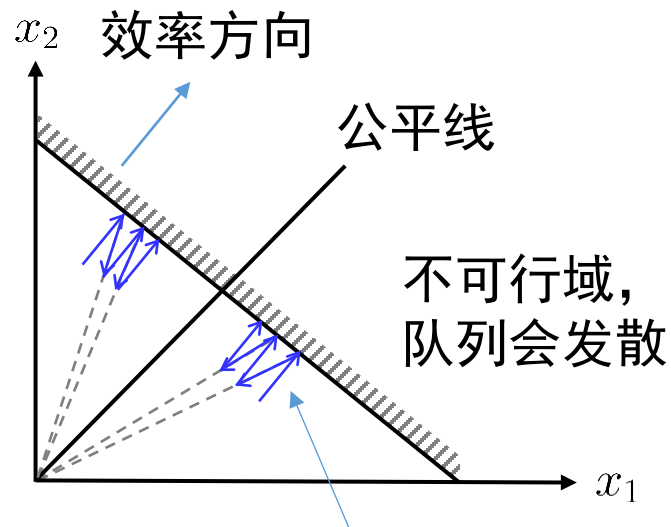
即不拥塞时 $x_i(t)$ 线性增长；

- ② $\sum_i x_i \geq r$ 时， $\frac{d}{dt}x_i(t) = p - qx_i(t) \approx -qx_i(t)$

即拥塞时， $x_i(t+1) - x_i(t) = -qx_i(t)$

$x_i(t+1) = (1-q)x_i(t)$ ， $x_i(t)$ 乘性减小（越大的减少越多）。

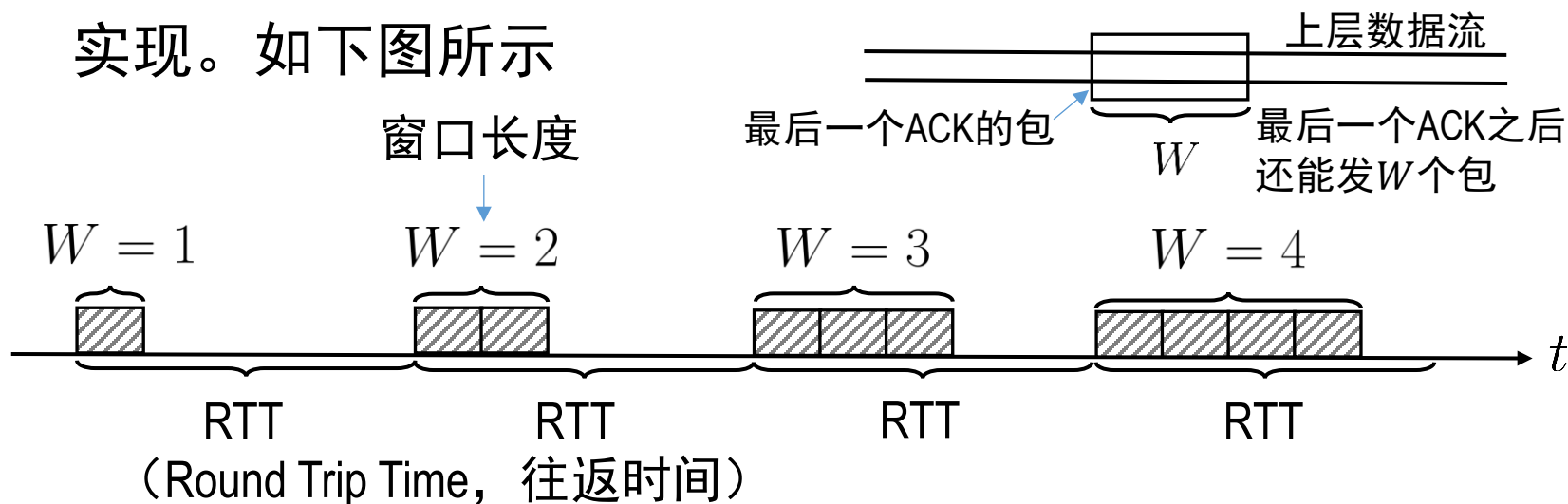
这个“加性增，乘性减”（Additive Increase, Multiplicative Decrease, AIMD）
算法对延时、反馈噪声等异常的鲁棒



几何解释：
“加性增” 平行于公平线
“乘性减” 延长线过原点

基于窗的端到端流量控制（一）

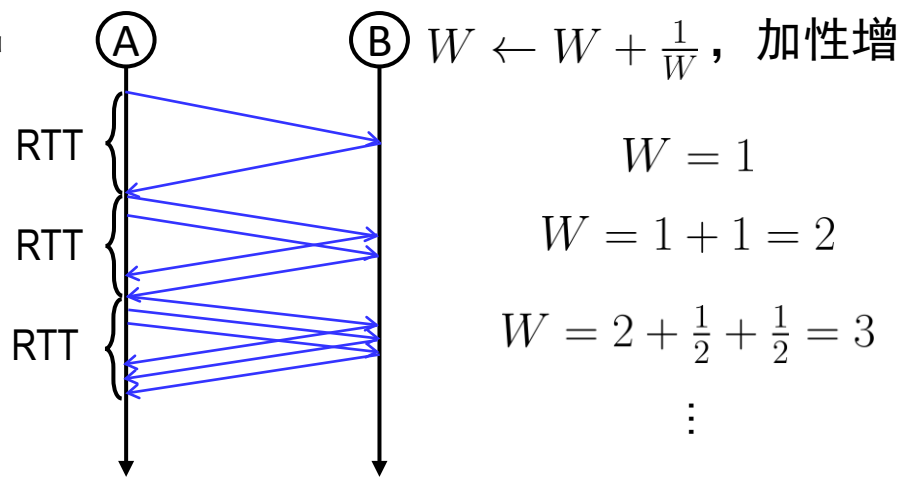
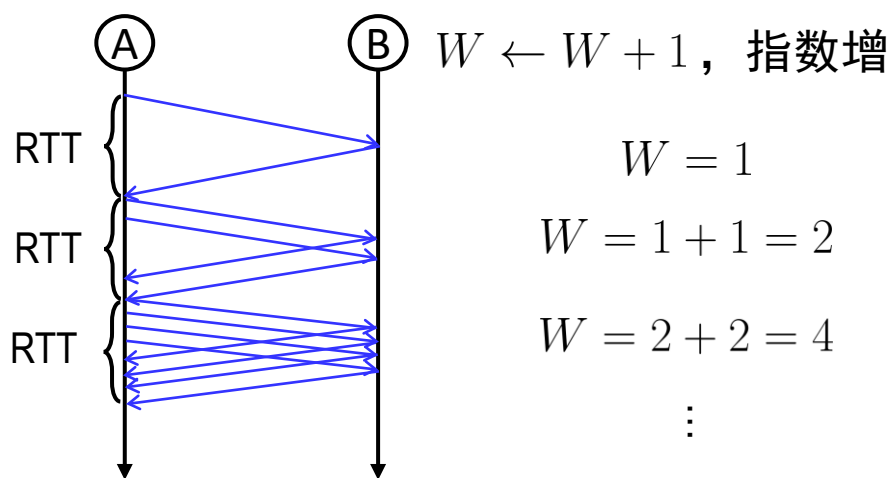
- 上述流量控制是基于流模型展开讨论，我们默认信源 i 的速率 $x_i(t)$ 可以连续（至少是以足够小的颗粒度）进行调整。而实际系统中，我们只能决定当前时刻发不发包，或隔多久发下一个包。这种对离散事件的控制，可通过引入窗来实现。如下图所示



- 不拥塞的情况下：RTT基本不变化，而窗口 W 内发的包基本在RTT时刻附近来返回ACK包，此时瞬时速率 $x(t) = \frac{W}{RTT} \times \text{每包比特数}$

基于窗的端到端流量控制（二）

- 由P9讨论知，基于窗的流量控制也要让 $x(t)$ 尽量保持在可行域（队列不发散）以内，一旦丢包，则窗口减半，此时更倾向于回到“不拥塞”状态，即近似让瞬时速率减半，故有“乘性减”（Multiplicative Decrease）：RTT超时门限内未收到ACK（判定丢包） $W \leftarrow \frac{W}{2}$ 。在“非拥塞”情况下，由于瞬时速率与窗长 W 近似成正比，故有“加性增”（Additive Increase）：每收到一个ACK， $W \leftarrow W + \frac{1}{W}$ ← 为何不是1？



基于窗的端到端流量控制（三）

- 上述AIMD方法中的窗长 W 的动态变化也可以由如下随机微分方程给出

$$\frac{dW(t)}{dt} = \frac{1}{RTT} - \frac{W(t)}{2} dv(t)$$

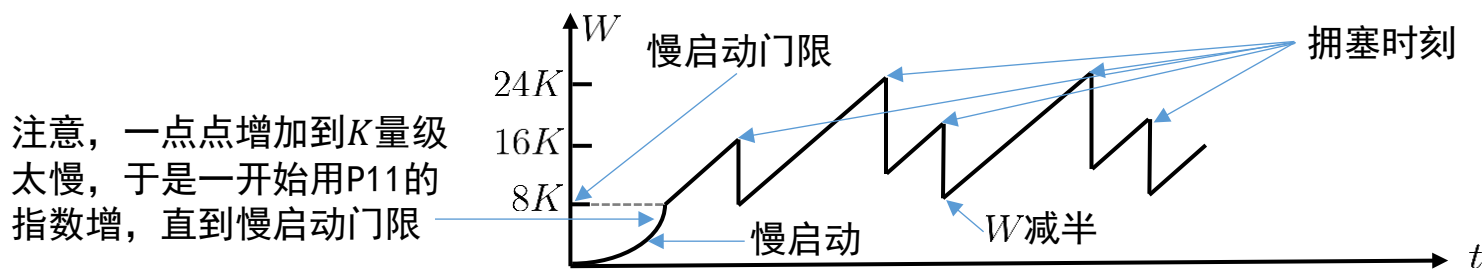
$$dv(t) = \mathbb{1}\{t \text{ 时刻检测到超时、丢包}\}$$

- 由此，我们还可以刻画拥塞节点 A 处队列长度为

$$\frac{dq(t)}{dt} = \frac{W}{RTT} N - r$$

总用户（流）数

- 上述模型应用于大量现代TCP研究（以流模型，控制模型为主）中。放眼整体，AIMD的窗口变化如下图所示：

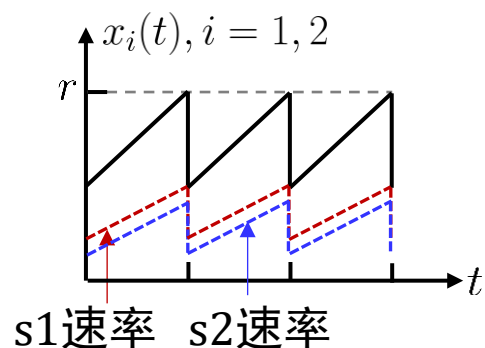


最大 W 的均值

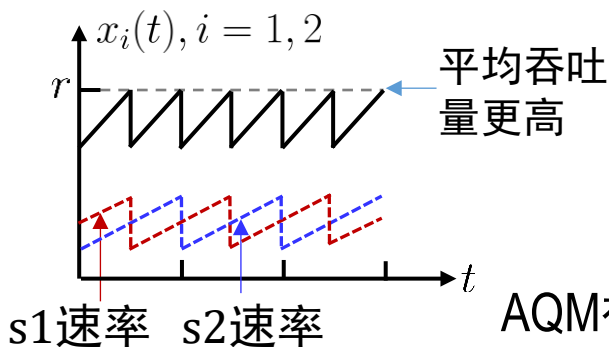
$$\text{吞吐量 } R = \frac{0.75 \bar{W}_{\max}}{RTT} \quad \left(\text{改进后可达 } \frac{1.2 \times \text{最大包长}}{RTT \sqrt{\text{丢包率}}} \propto \frac{r}{RTT \sqrt{\text{丢包率}}} \right)$$

基于窗的端到端流量控制（四）

- 上述端到端流量控制会产生一种“同步振荡”从而使平均吞吐量远小于系统可提供的带宽（即网络中速率） r ，如下左图所示



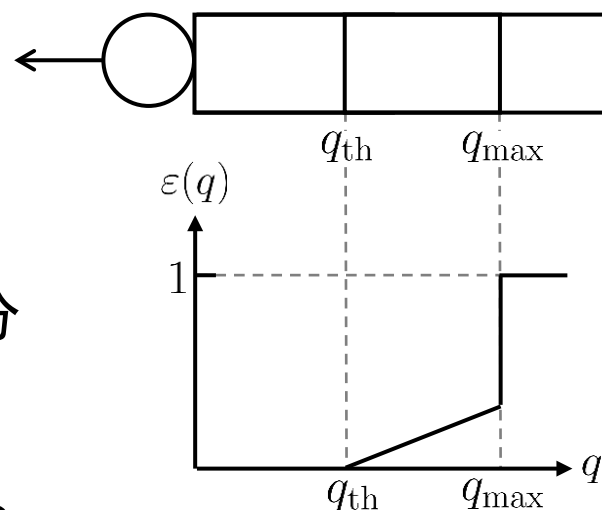
同步振荡



异步振荡

改进方案：主动队列管理（AQM），即在A的缓存中，当队列长度高于门限 q_{th} 后，就以概率 $\varepsilon(q)$ 随机丢弃分组，从而让个别用户提早减小窗口 W ，破坏掉同步振荡——RED

（Random Early Detection, 随机早期检测）



$$\varepsilon(q) = \begin{cases} \beta(q - q_{th})^+, & q < q_{max} \\ 1, & q \geq q_{max} \end{cases}$$

AQM被认为是流量控制的效用最大化问题（NUM）的拉格朗日对偶，其通过队长、丢包等参数更新通知用户“资源实时价格”，从而实现分布式主—对偶求解NUM

基于窗的端到端流量控制（五）

在上面的讨论中，RTT扮演了重要角色，应用于超时丢包判断、速率控制与传输等。但网络中的RTT也随拥塞状态、路径长度等改变。

① 直接平均法估计 $\overline{\text{RTT}}$ ，
$$\overline{\text{RTT}}[K] = \frac{1}{K} \sum_{i=1}^K \text{RTT}[i]$$

缺点：跟踪状态变化不敏捷，应给近期RTT更大权重 遗忘因子

② 指数加权平均法估计 $\overline{\text{RTT}}$ ，
$$\overline{\text{RTT}}[K] = \alpha \cdot \overline{\text{RTT}}[K-1] + (1-\alpha) \cdot \text{RTT}[K]$$
$$= (1-\alpha)\text{RTT}[K] + \alpha(1-\alpha)\text{RTT}[K-1] + \alpha^2(1-\alpha)\text{RTT}[K-2] + \cdots + \alpha^{K-1}(1-\alpha)\text{RTT}[1]$$

此时，一般令超时门限 $\text{RTO} = g \times \overline{\text{RTT}}$ （ g 常取2）

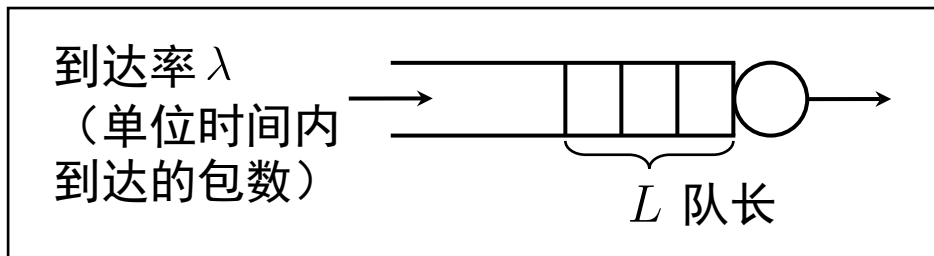
- ③ Jacobson等人考虑了RTT的偏差 $\Delta = \mathbb{E} \{ |\text{RTT} - \mathbb{E}\{\text{RTT}\}| \}$ 的影响，
若RTT为i.i.d. r.v. 则 $\Delta[1] = \frac{1}{2}\text{RTT}[1]$ ， $\Delta[K] = \frac{1}{K}\Delta[K-1] + \frac{1}{K} [\text{RTT}[K] - \overline{\text{RTT}}[K-1]]$
若考虑②的遗忘加权，则 $\Delta[K] = \beta\Delta[K-1] + \beta |\text{RTT}[K] - \overline{\text{RTT}}[K-1]|$
考虑RTT的偏差后，一般令超时门限 $\text{RTO} = \overline{\text{RTT}} + h \times \Delta$ （ h 常取4）

网络延时（一）

延时是网络服务质量（QoS, Quality of Service）的重要指标，延时分析常依赖概率模型如排队系统等，相关结论也适用于ARQ、多址等。

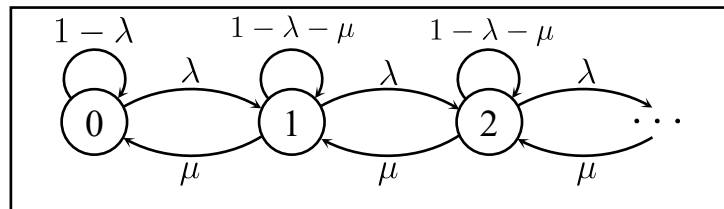
① Little定理：对任意排队

系统： $\bar{L} = \lambda W$
 $\uparrow$
 在队列中的平均时间



② M/M/1队列：到达为到达率 λ 的泊松过程，服务时间为均值 μ^{-1} 的指数分布，其马氏链如图。包平均

消耗于队列的时间 $W = \frac{1}{\mu - \lambda}$



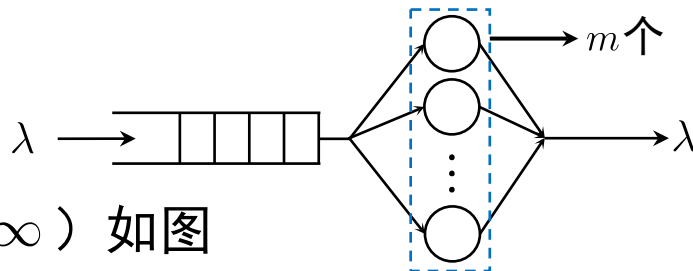
（这就回过头解释了统计复用的优势，因 $\frac{1}{\mu/N - \lambda/N} = \frac{N}{\mu - \lambda}$ ）

③ M/G/1队列：若服务时间为一般分布，均值仍为 μ^{-1} ，方差 σ^2 ，则由嵌入式马氏链分析，得Pollaczek-Kintchine公式

$$W = \frac{1}{\mu - \lambda} \times \frac{(1 + \mu^2 \sigma^2) \lambda + 2(\mu - \lambda)}{2\mu} \quad \text{特例M/D/1: } W = \frac{2\mu - \lambda}{2\mu(\mu - \lambda)}$$

网络延时（二）

回顾电平和波形信道，线性处理中高斯噪声可保持高斯性质为我们的讨论带来了极大的便利。在排队系统中，指数分布服务台也可让泊松到达保持泊松性质。



- ① Burke定理：M/M/ m ($m = 1, 2, \dots, \infty$) 如图

队列的离去过程仍为到达率 λ 的泊松过程，且 t 时刻系统中的包与 t 之前的离去时间独立。Burke定理确定了一个路由（ m 个服务台）的泊松不变性。如下定理则进一步将这一较强的结论推广到一般网络。

- ② Jackson定理：考虑一个 k 个节点组成的排队网络，节点 i 服务完的包以概率 θ_{ij} 去向节点 j （若图模型上 i, j 不邻接，则必有 $\theta_{ij} = 0$ ）记节点 i 的外部到达为到达率 a_i 的泊松过程，记 $\lambda_i = a_i + \sum_{j=1}^k \lambda_j \theta_{ji}$ 令 $\rho_i = \frac{\lambda_i}{\mu_i}$ ， μ_i 为节点 i 服务率，则有 $P(\mathbf{n}) = P_1(n_1)P_2(n_2) \cdots P_k(n_k)$ ， n_i 为节点 i 处排队包数， $P_i(n_i) = \rho_i^{n_i} (1 - \rho_i)$

（故Max-Flow-Min-Cut定理也可确定随机网络的最大稳定域）

网络延时（三）

在Burke定理和Jackson定理的基础上，我们不仅能给出网络能承载的业务量极限，还能通过对路由的优化最小化业务的延时，首先引入一些符号，我们记：（先考虑一个s-d对）从s到d的总业务量（流量）为 x_s ，可用路径 $p \in \{1, 2, \dots, K\}$ ，路径 p 上分配的流量 x_p （本质是负载均衡），网络中的链路（仅邻接节点间存在） $l \in \{1, 2, \dots, J\}$ ，链路 l 的容量 r_l 。这里，我们不使用二元数组表示链路，是为了后面的形式简洁。

定义：路径-链路映射矩阵 $\mathbf{H} = [h_{lp}]_{J \times K}$ ，其中

$$h_{lp} = \begin{cases} 1, & \text{若路径 } p \text{ 使用了链路 } l \\ 0 & \text{其他} \end{cases}$$

于是，链路 l 上承载的总到达率为

$$\lambda_l = \sum_{p=1}^K x_p h_{lp}, \quad l = 1, 2, \dots, J.$$

网络延时（四）

记路由分配矢量 $\mathbf{x} = [x_1, x_2, \dots, x_K]^T$ ，链路负载矢量 $\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \dots, \lambda_J]^T$ 因此有 $\boldsymbol{\lambda} = \mathbf{H}\mathbf{x}$ 。接下来，我们给出链路 l 上的延时代价，为：

$$D_l(\lambda_l) = \left(\underbrace{\frac{1}{r_l - \lambda_l}}_{\text{排队延时}} + \underbrace{\tau_l}_{\text{传播延时}} \right) \times \underbrace{\lambda_l}_{\text{业务量权重}}$$

于是，我们可以建立优化问题（最小延时路由）如下：

$$\begin{aligned} \min \quad & \sum_{l=1}^J \left(\frac{\lambda_l}{r_l - \lambda_l} + \lambda_l \tau_l \right) \\ \text{s.t.} \quad & \begin{cases} \boldsymbol{\lambda} = \mathbf{H}\mathbf{x} & \text{—— 路径-链路映射} \\ \mathbf{1}\mathbf{x} = x_s & \text{—— 总业务量约束} \\ \mathbf{x} \geq 0 & \text{—— 流量非负} \end{cases} \end{aligned}$$

若最优解 $\mathbf{x}^*$ 中只有一个分量非零（则即为 x_s ），此时采用单路径路由，否则为多路径路由。若 $x_s \ll r_l$ ，即 $\lambda_l \ll r_l$ ，则最优解为最小代价（最短路径）路由。由此知有线网（如光纤）中采用最短路径路由的合理性，而带宽受限的无线网络可通过多路径路由均衡负载，降低延时。

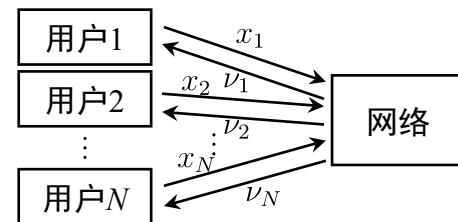
网络延时（五）

上面，我们处理了单一s-d对的最小延时路由问题（而且求解的不是一个0-1规划问题）。下面我们将其拓展到多个s-d对，并考虑多对s-d之间的速率公平和有效分配。此时可以把最小延时路由和网络效用最大化问题结合起来，得到：

总的s-d对数量 $\rightarrow$

$$\min \sum_{s=1}^N U(x_s) - \sum_{l=1}^J D_l(\lambda_l)$$

$$\text{s.t.} \begin{cases} \lambda = \sum_{s=1}^N \mathbf{H}_s \mathbf{x}_s & (\mathbf{H}_s \text{ 为用户 } s \text{ 的链路映射矩阵}) \\ \mathbf{1} \mathbf{x}_s = x_s & (\text{用户 } s \text{ 的业务量约束}) \\ \mathbf{x}_s \geq 0 \end{cases}$$



显然，这个问题是较为复杂的。先对每对s-d选最短路径，再AIMD流控，是一种启发式的求解方法。运筹学中常采用解耦（Decomposition）方法求解此类问题。网络自身通过路由（设定链路代价）、主动队列管理（AQM），在优化路径均衡负载的同时通知用户其使用资源的“价格”（服务 s 的路径 p 的最小边际成本）

$$\nu_s = \min_p \sum_{j \in \text{路径 } p} D_j(\lambda_j)$$

用户求解自身的效用最大化问题，即

$$\max U(x_s) - \nu_s x_s \quad \text{s.t. } x_s \geq 0$$

于是用户可根据网络反馈（丢包，延时等）独立完成端到端流控。

谢谢！

Thanks！